Praxisbericht zur Praxisphase

Entwicklung und Integration des FATTA Annotation Tools

Four Dimension Anatomic Topographic Annotation Tool

Name	Süheyl-Seref Aslan
Matrikelnummer	100703
Studiengang	Medieninformatik
Praktikumsstelle	Charité - Universitätsmedizin Berlin, Projektumfeld FATTA
Ausbildungsstelle / Bereich	Klinisches Forschungs- und Entwicklungsumfeld mit Fokus auf medizinische Bildannotation, Dokumentation und browserbasierte Werkzeugentwicklung
Praxisbetreuung	BHT: Prof. Dr.-Ing. Francisco Morales Serrano Charite: Dr. med. Jörg Neymeyer
Zeitraum / Umfang	15.03 - 07.06. 2026
Datum der Abgabe	10.07.2026

Inhaltsverzeichnis

1 Kurzbeschreibung der Ausbildungsstelle 3
1.1 Projektumfeld und technische Ausgangslage 4
2 Aufgabenbeschreibung 4
3 Beschreibung der Lösung der praktischen Aufgaben 6
3.1 Anforderungsanalyse und Zielbild 6
3.2 Datenmodell und Rückgabeformat 7
3.3 Opener-to-Tool-Integration 8
3.4 Zeichnen, Auswählen und Speichern von Annotationen 8
3.5 Background-Segmente und räumliche Zuordnung 9
3.6 Laufende Nummerierung der Annotationen 10
3.7 Summary, lesbare Rückgabe und Proof-Bild 11
3.8 Spezialisierte Background-Erstellung 11
3.9 Benutzeroberfläche, Help und Branding 12
3.10 Tests, Debugging und Qualitätssicherung 12
3.11 Übergabe an Model View und Ausblick auf Skalierung 13
4 Bezug zum Studium und eingesetzte Kompetenzen 13
5 Bewertung des Praktikums und Reflexion 14

1 Kurzbeschreibung der Ausbildungsstelle

Die Praxisphase habe Ich im Projektumfeld der Charité - Universitätsmedizin Berlin durchgeführt. Die Charité zählt zu den größten universitätsmedizinischen Einrichtungen Europas und verbindet Krankenversorgung, Forschung und Lehre. Das hier bearbeitete Projekt war in einem klinischen Forschungs- und Entwicklungsumfeld angesiedelt in dem browserbasierte Softwarewerkzeuge zur strukturierten medizinischen Dokumentation, zur Bildannotation und zur späteren datengetriebenen Auswertung entwickelt und erprobt werden.

Der fachliche Kontext des Praktikums ist die Entwicklung eines webbasierten Annotationstools mit dem Arbeitstitel FATTA. FATTA steht hier für “Four Dimension Anatomic Topographic Annotation Tool”. Das Werkzeug dient dazu medizinische Bildinformationen nicht nur als Bildmarkierungen, sondern auch strukturiert und maschinenlesbar abzulegen. Dadurch können Befunde, Regionen, Positionsinformationen, manuelle Beschreibungen und automatisch abgeleitete Zuordnungen in einem einheitlichen JSON-basierten Datenformat gespeichert werden.

Der praktische Nutzen des Werkzeugs ergibt sich daraus, dass medizinische Dokumentationen häufig visuell erfolgen, die visuelle Information aber ohne Struktur schwer vergleichbar und nur eingeschränkt automatisiert auswertbar machen. Das Projekt soll dieses Problem adressieren, indem es Annotationen, Koordinaten, Flächen, Labels und Beschreibungen in einem browserunabhängigen Workflow zusammenführt. Das Tool soll aber sowohl für klassische klinische Dokumentationssituationen als auch für Forschungsdemonstratoren, Notfalldokumentation und perspektivisch für die Zuordnung auf anatomische Avatar- oder Modellansichten verwendbar sein.

Die Arbeit an dem Projekt fand in einem engem Austausch mit der Praxisbetreuung statt. Anforderungen wurden nicht ausschließlich zu Beginn festgelegt, sondern iterativ anhand von Versuchen, Besprechungen und Rückmeldungen weiterentwickelt. Dadurch ähnelte der Ablauf einem praxisnahen Entwicklungsprozess: technische Zwischenstände wurden umgesetzt, getestet, mit dem Betreuer diskutiert und anschließend funktional und technisch angepasst.

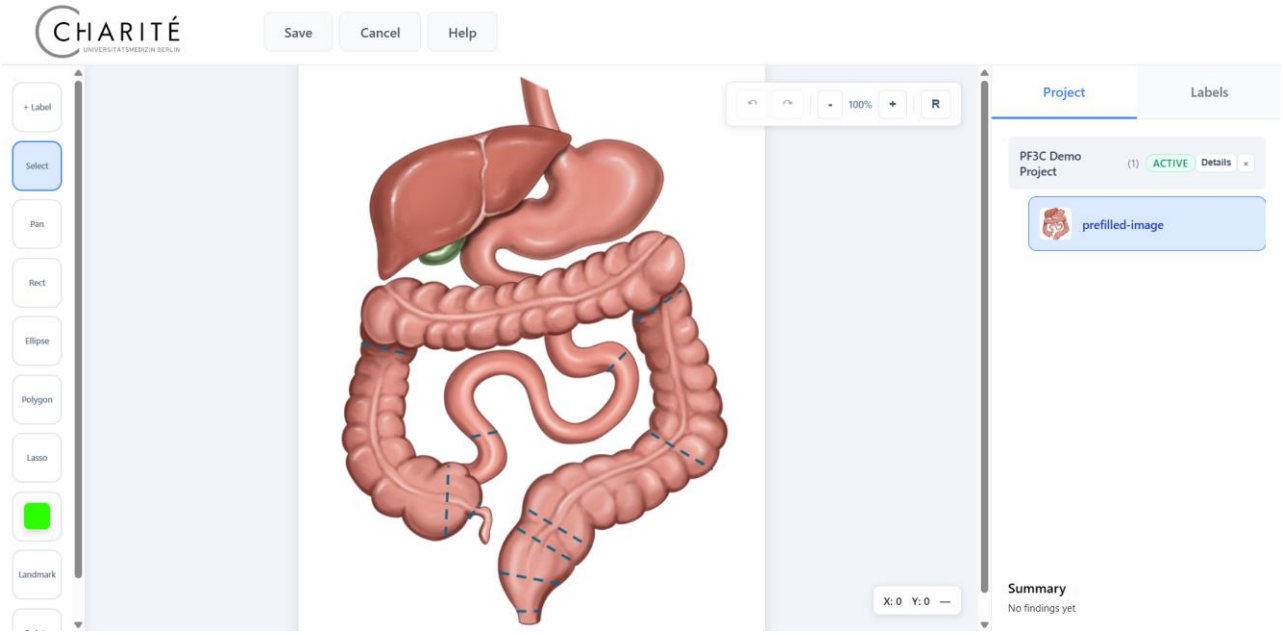


Abbildung 1: Benutzeroberfläche des FATTA Annotation Tools.

1.1 Projektumfeld und technische Ausgangslage

Technisch basiert das Werkzeug auf einer lokalen webbasierten Anwendung. Die Oberfläche der Anwendung besteht aus HTML, CSS und JavaScript und nutzt einen Canvas zur Darstellung des Bildes und der dazugehörigen gezeichneten Annotationen. Die Anwendung kann über ein aufrufendes Formular geöffnet werden. Dieses sogenannte Opener-to-Tool- bzw. Tool-to-Opener-Prinzip war für die Einbindung in das bestehende System wesentlich, weil das Tool dadurch wie ein eigenständiges Modul gestartet und seine Ergebnisse wieder an das ursprüngliche Formular zurückgeben kann.

Die Anwendung verwendet mehrere getrennte Module. Dazu gehören unter anderem Komponenten für Canvas-Management, Zeichenwerkzeuge, Projektverwaltung, Labelverwaltung, Analyse und Rückgabe an den Opener. Diese modulare Struktur erleichterte die Erweiterung, da Ich neue Funktionen wie Background-Segmente, Summary-Rückgabe, Proof-Bild-Export oder Annotation-Nummern an definierten Stellen ergänzt werden konnten.

Ein besonderer Aspekt war, dass die Lösung ohne komplexe Serverabhängigkeiten funktionieren sollte. Viele Versuche erfolgten lokal über einen einfachen Webserver. Diese Architektur ist für medizinische Notfall- und Demonstrationsszenarien relevant, da die Anwendung auch in einer lokalen Umgebung und unabhängig von einer zentralen Internetverbindung lauffähig sein soll.

2 Aufgabenbeschreibung

Die Aufgaben während des Praktikums bezogen sich auf die Weiterentwicklung eines vorhandenen webbasierten Annotationstools zu einem strukturierten medizinischen Dokumentationswerkzeug. Der Schwerpunkt lag nicht nur auf dem Zeichnen von Formen, sondern insbesondere auf der

zuverlässigen Speicherung, Rückgabe, Auswertung und Wiederverwendung der erzeugten Annotationen.

Zu Beginn stand ein funktionsfähiger Grundstand des Tools zur Verfügung. Darauf aufbauend wurden Anforderungen in mehreren Iterationen konkretisiert und erarbeitet. Die Zielrichtung war aus einer einfachen Bildannotation eine eindeutige, nachvollziehbare und später auswertbare Datenstruktur zu erzeugen. Dabei mussten sowohl Bedienbarkeit des Werkzeugs als auch das Datenmodell und die Rückgabeformate berücksichtigt werden.

Aufgabenbereich	Konkrete Tätigkeit	Technische Umsetzung	Ergebnis / Nutzen
Analyse der bestehenden Anwendung	Sichtung der vorhandenen HTML-, CSS- und JavaScript-Struktur, Prüfung der Datenflüsse und Identifikation fehlender Rückgabefelder.	Debugging im Browser, Untersuchung von CanvasManager, labelManager, openerBridge, analysisSA und Background-Dateien.	Grundlage für gezielte Erweiterungen und Fehlerbehebung.
Opener-Workflow	Einrichtung und Stabilisierung der Übergabe von Import- und Return-Daten zwischen Testseite und Tool.	PROJECT_IMPORT_JSON, RETURN_ANNOTATION_JSON, RETURN_SUMMARY_JSON, RETURN_SUMMARY_TEXT und RETURN_EXPORT_IMAGES_BASE64.	Das Tool kann als Modul aus einem Formular geöffnet und wieder zurückgegeben werden.
Background-Segmente	Laden bildbezogener polygon_descriptions-Dateien und Zuordnung gezeichneter Formen zu anatomischen Bereichen.	picturePolygonDescriptions.js und analysis.js mit Polygon-/Form-Relationen.	Automatische areaIds und lesbare analysisText-Informationen.
Annotation-Nummerierung	Eindeutige Nummern für gezeichnete Annotationen erzeugen, anzeigen und exportieren.	number-Feld pro Annotation, Toggle Show Annotation Numbers, Canvas-Rendering und Export-Rendering.	Eindeutige Zuordnung zwischen Form, Summary und exportiertem Bild.
Summary und Export	Separater lesbarer und maschinenlesbarer Ergebnisexport.	buildSummaryText, buildSummaryResultJSON, saveAndClose und openerBridgeSA.	Direkte Lesbarkeit für Anwender und strukturierte Weiterverarbeitung.
Background-Editor	Erstellung einer spezialisierten Seite zur Erzeugung der polygon_descriptions-Dateien.	Backgrounderstellungstestseite mit JSON- und Proof-Bild-Download.	Einmalige Segmentierung eines Bildes kann nachvollziehbar gespeichert werden.
UI/UX und Hilfe	Verbesserung der Bedienbarkeit, Help-	Help-Seite, FATTA-Logo, Label-Settings, stabilisierte Buttons.	Besserer Demo- und Anwenderfluss.

	Button, Branding und Fehlerbehebung.		
Vorbereitung Weiterverarbeitung	Konzept für Übergabe an Model-View/Avatar-Seite und spätere Skalierung.	sessionStorage-Handoff, Model-View-Demo und Auto-/Worker-Konzept.	Vorbereitung einer späteren 2D-/3D-Visualisierung.

Tabelle 1: Zusammenfassung der wesentlichen Aufgabenbereiche während der Praxisphase.

3 Beschreibung der Lösung der praktischen Aufgaben

3.1 Anforderungsanalyse und Zielbild

Die fachliche Anforderung der Aufgaben bestand darin, medizinische Bildinformationen so zu erfassen, dass sie nicht nur visuell sichtbar, sondern auch eindeutig weiterverarbeitbar sind. Eine reine Bilddatei mit eingezeichneten Markierungen reicht dafür nicht aus da eine nachträgliche Analyse, Suche oder Vergleichbarkeit über Zeitpunkte nur eingeschränkt möglich wäre. Deshalb sollte jede Annotation als strukturierter Datensatz gespeichert werden.

Das Zielbild war ein Workflow in dem ein Anwender/User ein Bild lädt, Labels auswählt, Formen einzeichnet kann, manuelle Beschreibungen ergänzt kann und anschließend ein vollständiges maschinenlesbares Ergebnis erhält. Desweiteren sollte das Ergebnis für Menschen verständlich bleiben und daher wurden neben dem vollständigen JSON auch eine lesbare Summary und ein exportiertes Bild mit sichtbaren Markierungen vorgesehen um das zu garantieren.

Ein wichtiger technischer Grundsatz war die Trennung von Erfassung und späterer Visualisierung. Das Annotationstool soll die Daten zuverlässig erfassen und exportieren. Eine spätere Darstellung auf einem anatomischen Avatar, einem Model-View oder einem WebGL-System kann darauf aufbauen, ohne das Grundformat zu verändern.

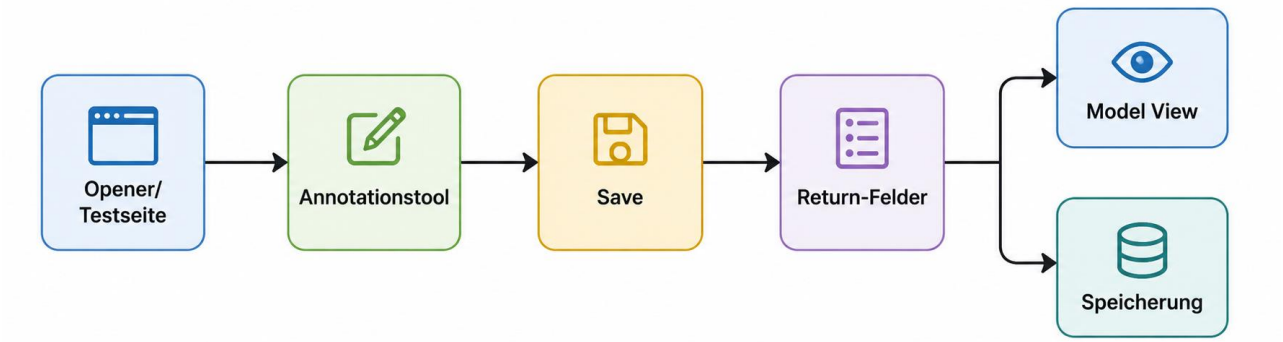


Abbildung 2: Datenfluss des Opener-to-Tool-Workflows.

3.2 Datenmodell und Rückgabeformat

Ein zentraler Arbeitsschritt war die Stabilisierung des Datenmodells. Die Anwendung arbeitet mit Projekten, Bildern, Labels und Annotationen. Das Projekt enthält Metadaten und eine Bildliste. Jedes Bild enthält Annotationen jede Annotation besitzt eine ID, ein Label, einen Typ, Koordinaten und zusätzliche fachliche Felder wie areaIds, number, subdescriptionManual, analysisText und displayText.

Die ID identifiziert eine Annotation technisch. Das Feld number dient dagegen als sichtbare laufende Nummer für die Anwender. Diese Unterscheidung ist wichtig, weil technische IDs lang und nicht gut lesbar sind, während laufende Nummern in Besprechungen, Screenshots und Summary-Texten direkt verwendet werden können und lesbarer sind.

Das vollständige Ergebnis wird in dem Feld RETURN_ANNOTATION_JSON zurückgegeben. Dieses JSON enthält die Daten so, dass das Projekt später wieder geöffnet oder maschinell weiterverarbeitet werden kann. Zusätzlich wurden Felder wie RETURN_SUMMARY_JSON und RETURN_SUMMARY_TEXT ergänzt. Das strukturierte Summary-JSON eignet sich für Programme, während der Summary-Text für Anwender unmittelbar lesbar ist. Dadurch muss ein Anwender nicht zuerst ein JSON durchsuchen, um dann die wesentlichen Ergebnisse zu verstehen.

Beim Speichern soll vor der Rückgabe die Analysefunktion ausgeführt werden. Dadurch ist sichergestellt, dass Felder wie areaIds, number, displayText und areaSummary vor dem Export aktuell sind. Diese Reihenfolge war ein wichtiger Fehlerbehebungsschritt weil asynchrone Analysefunktionen sonst dazu führen können, dass sichtbare Informationen im Tool vorhanden sind, aber im zurückgegebenen JSON fehlen.

3.3 Opener-to-Tool-Integration

Die Einbindung in eine übergeordnete Anwendung erfolgt über ein Formular im aufrufenden Fenster. Dieses Formular enthält Eingabe- und Rückgabefelder. Das Annotationstool liest beim Start PROJECT_IMPORT_JSON aus dem Opener und schreibt beim Speichern die Return-Felder zurück. Dieses Prinzip habe Ich im Praktikum mit einer Testseite simuliert.

Die technische Herausforderung lag darin, dass das Tool in einem separaten Browserfenster läuft. Es muss jedoch zuverlässig auf das aufrufende Fenster zugreifen können. Dafür wurde eine OpenerBridge verwendet, die die Felder im MAINFORM des Opener-Fensters liest und beschreibt. Der Vorteil dieses Ansatzes liegt darin, dass er sehr gut zu formularbasierten Bestandsanwendungen passt.

Neben dem vollständigen Annotation-JSON wurde auch ein Base64-Bildexport hinzugefügt. Dieses exportierte Bild enthält die sichtbaren Markierungen und Nummern. Es kann als Nachweis oder als Preview in einer nachgelagerten Dokumentation verwendet werden. Damit wird der Brückenschlag zwischen maschinenlesbarer Datenstruktur und von Anwender prüfbarer Darstellung hergestellt.

3.4 Zeichnen, Auswählen und Speichern von Annotationen

Die Benutzerinteraktion findete hauptsächlich auf einem HTML Canvas statt. Über die Toolbar können verschiedene Werkzeuge ausgewählt werden, darunter Rechteck, Ellipse, Polygon, Lasso, Landmark und Delete optionen. Eine Annotation kann dadurch entweder als einfache geometrische Form oder als freie Polygonform erfasst und dargestellt werden.

Während der Umsetzung musste darauf geachtet werden, dass unterschiedliche geschlossene Formen in der Analyse berücksichtigt werden. Nicht nur Polygonformen, sondern auch Rechtecke, Ellipsen und Freihandformen müssen räumlich mit Background-Segmenten verglichen werden können. Deshalb ist eine Funktion zur Normalisierung von Annotationen in Polygonpunktfolgen eingesetzt worden. Rechtecke werden in vier Eckpunkte umgewandelt, Ellipsen durch mehrere Punkte und Freehand-Formen bei sehr vielen Punkten sinnvoll reduziert.

Ein weiterer praktischer Punkt war die Vermeidung von unbeabsichtigten Zusatzannotationenen. Die eingeführte Doppelklickfunktion zur Beschreibungseingabe führte dazu, dass kleine Ghost-Shapes entstanden sind, was nicht der Fall sein sollte. Solche Detailprobleme zu entdecken war für die fachliche Qualität und Anwendung wichtig, weil zusätzliche unsichtbare oder sehr kleine Formen sonst Summary und Export verfälschten.

Beim Löschen und beim Aktualisieren von Labels mussten Canvas-Zustand und Projektdaten synchron gehalten werden. Eine Annotation die Erstellt wurde darf nicht nur visuell aus dem Canvas verschwinden, sondern muss auch aus der Annotation-Liste entfernt werden. Umgekehrt müssen Änderungen an Label-Eigenschaften wie Füllung, Rahmen oder Nummernanzeige sofort in der Canvas-Darstellung sichtbar werden.

3.5 Background-Segmente und räumliche Zuordnung

Ein wesentlicher fachlicher Schritt war die Einführung von Background-Segmenten. Dabei wird zu einem Bild eine separate Datei mit Polygon-/Annotationbeschreibungen abgelegt. Der Dateipfadname orientiert sich am Bildnamen. Zu einem Bild test.svg gehört beispielsweise eine Datei test.json im Ordner polygon_descriptions. Diese Datei enthält Segment-IDs, Titel und Punktfolgen.

Die Analysefunktion lädt die passende Background-Datei für das aktuell geladene Bild. Anschließend wird jede gezeichnete Annotation mit den hinterlegten Segmenten verglichen was zu Ergebnissen wie areaIds und analysisText Rückgaben führt. Die areaIds Rückgabe enthält die technischen Segmentnummern, analysisText enthält die lesbare Benennung der betroffenen Bereiche und auf dieser Weise wird aus einer freien Markierung eine fachlich verortete Annotation.

Bei der Prüfung wird die geometrische Beziehung zwischen Annotation und Background-Polygon klassifiziert. Relevante Beziehungen sind zum Beispiel die Überschneidung, vollständiges Enthaltensein oder Berührung. Dadurch muss eine Markierung nicht vollständig inerhalb eines Segments liegen, um als Treffer erkannt zu werden. Das entspricht der praktischen Arbeit, weil medizinische Befunde häufig Segmentgrenzen schneiden.

Die Background-Segmente sind vom normalen Annotationsergebnis getrennt und damit beschreiben werden die Hintergrundkarten eines Bildes, während die eigentlichen Findings durch den Anwender eingezeichnet werden. Diese Trennung wurde gameacht denn das macht das System flexibler, weil ein Bild einmal segmentiert und anschließend beliebig oft für neue Befundannotation verwendet werden kann.

3.6 Laufende Nummerierung der Annotationen

Die laufende Nummerierung wurde umgesetzt, um die Zuordnung zwischen Bildmarkierung, Summary und Datenexport zu verbessern. Jede gezeichnete Finding-Annotation erhält eine stabile Nummer. Diese Nummer ist nicht mit der technischen ID identisch, sondern dient der menschlichen Orientierung.

Die Nummern werden während der Analyse vergeben und bereits vorhandene Nummern bleiben erhalten, damit sich beim erneuten Speichern die Reihenfolge nicht unkontrolliert ändert. Neue Annotationen erhalten dadruch die nächste freie Nummer und dann kann ein Anwender eine Form im Bild sehen, die Nummer in der Summary finden und die gleiche Nummer im exportierten Bild wiedererkennen.

Für die Anzeige wurde ein Label-Setting Show Annotation Numbers eingeführt. Dieses Settingfunktion funktioniert ähnlich wie die schon davor vorhanden gewesenen Show Fill oder Show Border Funktionen. Wenn es aktiv ist, zeichnet der CanvasManager die Nummer direkt in die Mitte der jeweiligen Form und zusätzlich wird die Nummer beim Exportbild gerendert, sodass auch das zurückgegebene Bild die Nummern enthält.

Die Nummerierung wird außerdem in die Rückgabeformate integriert und sie erscheint im RETURN_ANNOTATION_JSON, im RETURN_SUMMARY_JSON, im RETURN_SUMMARY_TEXT und in der Project Summary des Tools. Damit ist die Nummer über alle Ebenen hinweg konsistent gehalten.

3.7 Summary, lesbare Rückgabe und Proof-Bild

Neben dem vollständigen JSON wurde eine zusammenfassende Ergebnisdarstellung erstellt. Die Project Summary im Tool zeigt direkt, welche Findings vorhanden sind, welche Nummer sie haben, welches Label zugeordnet ist und welche Background-Areas getroffen wurden sind. Die Darstellung soll beim Prüfen der Ergebnisse vor oder nach dem Speichern helfen.

Für die Rückgabe wurden zwei zusätzliche Felder ergänzt uzwar RETURN_SUMMARY_JSON, welche eine strukturierte Liste der Findings sowie Metadaten zur areaSummary enthält, und RETURN_SUMMARY_TEXT, die eine kompakte menschenlesbare Fassung enthält. Die Textversion ist besonders nützlich für Berichte, schnelle Kontrollen oder formularbasierte Dokumentationen.

Zusätzlich wird ein exportiertes Bild erzeugt welches das Originalbild mit den sichtbaren Annotationen, Markierungen und Nummern enthält. Für die Background-Erstellung ist dieses Proof-Bild besonders, weil man später nachvollziehen soll, welche Segmente mit welcher Nummer gezeichnet wurde. Dazu kann geprüft werden, ob die Bezeichnung in der polygon_descriptions-Datei zur tatsächlich gezeichneten Fläche passt oder nicht.

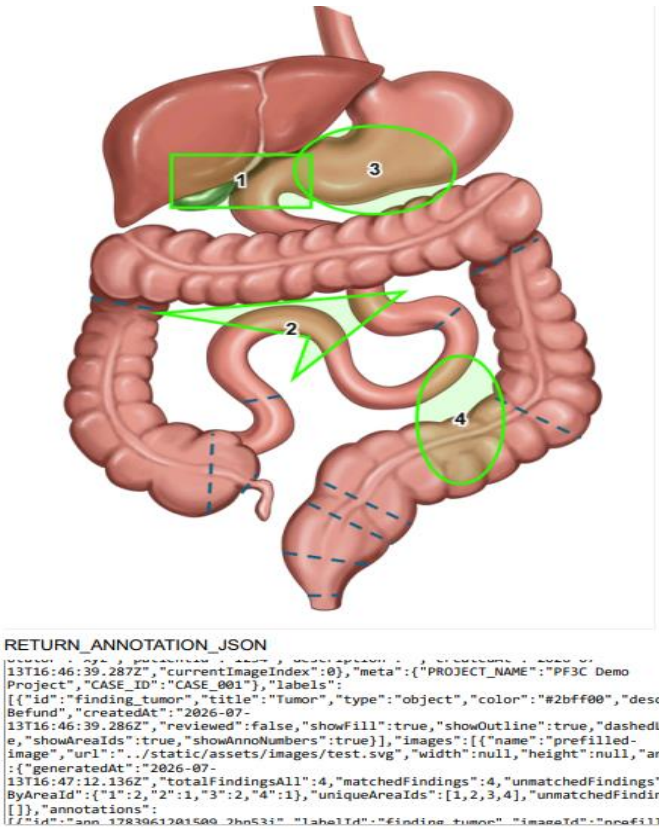


Abbildung 3: Exportiertes Bild mit sichtbaren Annotationen und Nummern

3.8 Spezialisierte Background-Erstellung

Für die einmalige Erstellung von Background-Segmenten wurde eine zum Testen basierte Backgrounderstellungstestseite angelegt. Diese Seite öffnet das Annotationstool mit einem vordefinierten Background-Label und der Anwender zeichnet die Segmentpolygone und speichert sie. Anschließend erzeugt die Seite aus den zurückgegebenen Annotationen eine polygon_descriptions JSON-Datei zum definieren.

Diese Seite unterscheidet sich von der normalen Test-Opener dadurch, dass sie nicht für beliebige Befundannotation gedacht sein soll, sondern gezielt für die Erstellung der Hintergrundkarte/-Segmenten eines Bildes. Das Ergebnis kann dann anschließend im Ordner static/assets/polygon_descriptions abgelegt werden. Beim nächsten Öffnen desselben Bildes kann das Tool diese Segmente automatisch laden und verwenden.

Zusätzlich wurde für diese Seite der Download eines Proof-Bildes vorgesehen damit die segmentierte Grundlage dokumentiert werden kann. Der Ansatz soll der Anforderung folgen, nicht zu viel Energie in einen komplexen Editor zu investieren, sondern einen funktional ausreichenden und nachvollziehbaren Workflow zu schaffen.

3.9 Benutzeroberfläche, Help und Branding

Im Verlauf des Praktikums es mehrere UI-Probleme die behoben werden mussten und dazu gehörten unter anderem nicht benötigte Felder, fehlerhafte Label-Speicherung, störende Fallback-Texte in der Hover-Anzeige sowie ein Problem, bei dem ein Label-Panel nach dem Aktivieren eines Toggles geschlossen wurden, Erstellungen von ungewollten Bescheibungen etc. Solche Änderungen wirken oberflächlich klein, sind aber für die Benutzbarkeit entscheidend.

Der Help-Button wurde hinzugefügt und die Help-Seite ist erstellt und angepasst worden, damit klarer gegliedert und mit Abschnitten zu Zielstellung, Kurzanleitung, Werkzeugen, Labels, Notes, Background-Segmenten, Speicherung und Troubleshooting sowie Kontribution ausgebaut. Außerdem wurde das FATTA-Branding mit Icon/Logo berücksichtigt.

Die Anwendung ist auf den Namen FATTA ausgerichtet, dazu wurden Titel, Favicon, Charité- und FATTA-Logo eingebunden. Das ist für Demonstrationen und die spätere Weitergabe wichtig, da das Tool dadurch nicht wie ein generisches Annotationstool wirkt, sondern als eigenständiges Werkzeug wiedererkennbar ist und auch die Entwicklerfirma damit repräsentiert wird.

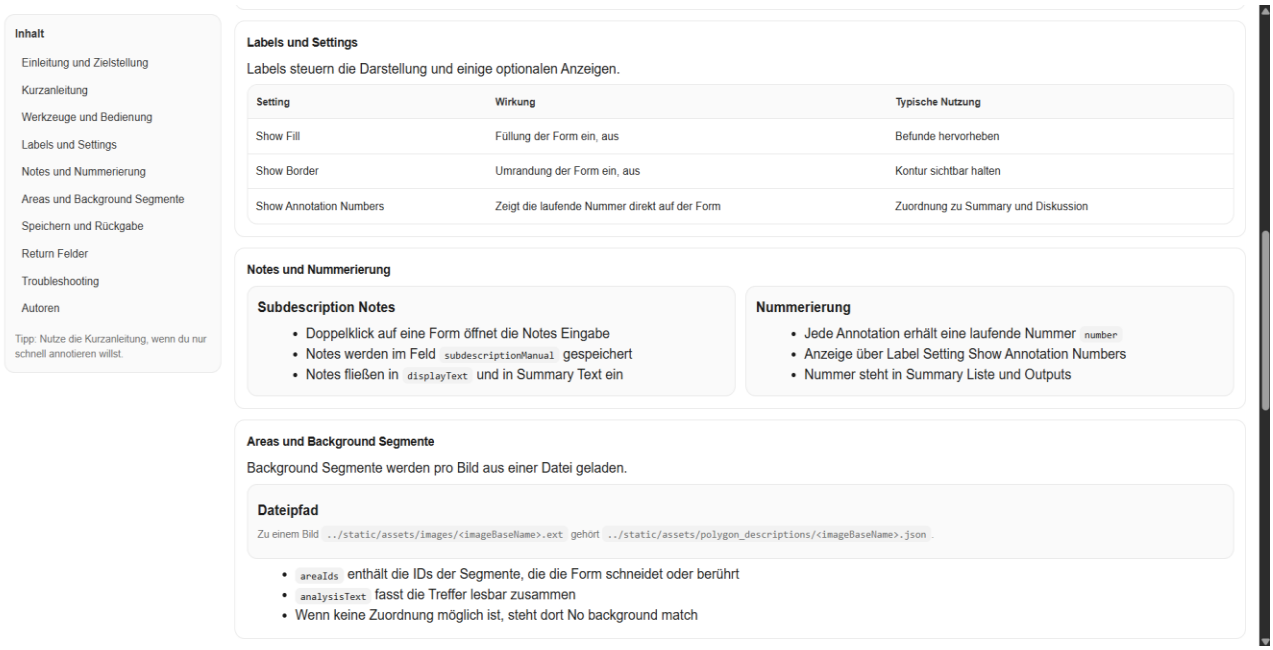


Abbildung 4: Auschnitt aus der Help-Seite des FATTA Tools.

3.10 Tests, Debugging und Qualitätssicherung

Die Qualitätssicherung erfolgte vor allem durch manuelle Tests im Browser, Debug-Ausgaben in der Konsole, dem Verwenden des Google-DevTools und wiederholte Testläufe mit unterschiedlichen Anwendungen. Dabei wurde geprüft, ob die Annotationen im Tool sichtbar sind, im Projektobjekt korrekt gespeichert werden, im Return-JSON erscheinen und im exportierten Bild gerendert werden.

Typische Testfälle waren das Zeichnen mehrerer Formen, das Hinzufügen von Notes per Doppelklickfunktion, das Aktivieren und Deaktivieren von Fill, Border und Annotation Numbers,

das Speichern in den Opener, das Prüfen der Summary-Felder und der Export eines Base64-Bildes. Bei Background-Segmenten wurde zusätzlich getestet, ob areaIds und analysisText passend zu den hinterlegten Segmenten entstehen.

Ein wichtiges praktisches Problem war der Browser-Cache da JavaScript- und JSON-Dateien lokal und statisch geladen werden, konnten alte Versionen im Cache bleiben was zu problem führte. Daher wurden Hard Reloads und gezielte DevTools-Prüfungen eingesetzt um diese Probleme zu umgehen. Dieses Problem zeigte, wie wichtig reproduzierbare Tests und klare Dateipfade in browserbasierten Anwendungen eigendlich sind.

Zur Fehlersuche wurden einzelne Funktionen isoliert getestet, Beispiel dafür sind loadForCurrentImage für Background-Segmente, runAnalysisForCurrentImage für die Berechnung von areaIds und number, buildSingleImageResultJSON für den Export sowie writeToOpener für die Rückgabe in das Formular. Diese modulare Debugging-Strategie soll eine systematische Eingrenzung von Fehlern ermöglichen.

3.11 Übergabe an Model View und Ausblick auf Skalierung

Ein weiterer Arbeitsschritt im Praktikum war die Vorbereitung eines nachgelagerten Handoff-Workflows bei dem die Idee ist, dass nach dem Speichern nicht nur Daten zurückgegeben werden, sondern eine nachgelagerte Seite diese Daten übernehmen kann. Dazu wurde eine Model-View-Demo geplant, die Return-Daten über sessionStorage erhalten und anzeigen kann.

Dieser Schritt ist als Vorbereitung auf spätere Skalierungs- oder Avatar-Workflows gedacht. Das Annotationstool selbst bleibt für die Erfassung zuständig und eine separate Seite soll Koordinaten transformieren, die Daten auf ein anderes Bild oder Modell übertragen und anschließend visualisieren können. Diese Trennung besteht, weil das Kernwerkzeug dadurch stabil bleibt und die spätere 2D-/3D-Visualisierung unabhängig bearbeitet und weiterentwickelt werden kann.

Für eine spätere 3D-Erweiterung ist das Datenmodell grundsätzlich erweiterbar. Zu den bestehenden X- und Y-Koordinaten könnte ein Z-Wert oder ein Tiefenbereich ergänzt werden. Dabei sollte beachten werden, dass die Performance bei vielen Punkten oder hohen Auflösungen begrenzt ist. Für eine browserbasierte WebGL-Darstellung müssten Daten gegebenenfalls reduziert oder skaliert werden.

4 Bezug zum Studium und eingesetzte Kompetenzen

Während des Praktikums kamen mir verschiedene im Studium erworbene Kompetenzen zur Anwendung vor. Dazu gehörten insbesondere Programmierung mit JavaScript, strukturierter Umgang mit Datenformaten, modulare Softwareentwicklung, Debugging, Grundlagen von Webtechnologien und konzeptionelles Denken in Datenflüssen. Was alles Konzepte aus Modulen wie Webengineering, Softwareengineering, und ähnlichen Modulen aus dem Studiengang sind.

Aus softwaretechnischer Sicht war das Projekt ein gutes Beispiel für die Arbeit an einer bestehenden Codebasis. Nicht jede Funktion sollte Ich neu entwickelt, sondern vielmehr mussten

Ich vorhandene Strukturen verstehen, erweitern und stabilisieren. Dabei war es erforderlich, Seiteneffekte zu erkennen. Eine Änderung an einer UI-Komponente konnte zum Beispiel Auswirkungen auf Speichern, Rendering oder Export haben.

Ziemlich relevant war auch der Zusammenhang zwischen Benutzeroberfläche und Datenmodell. Die sichtbare Form im Canvas ist nur dann gut nutzbar, wenn sie im Datenmodell eindeutig identifizierbar, speicherbar und wieder auslesbar ist. Dieser Zusammenhang entspricht grundlegenden Prinzipien der Informatik, wie etwa Datenmodellierung, Schnittstellendesign und Zustandsverwaltung.

Auch überfachliche Kompetenzen musste eingesetztet werden, darunter gehörten Kommunikation mit der Praxisbetreuung, Dokumentation von Zwischenergebnissen, strukturiertes Nachfragen bei unklarheiten und unklaren Anforderungen und die Fähigkeit, komplexe fachliche Vorstellungen in umsetzbare technische Teilaufgaben zu übersetzen. Gerade im medizinischen Kontext war es wichtig, technische Entscheidungen verständlich zu erklären und mit den fachlichen Anforderungen abzugleichen.

5 Bewertung des Praktikums und Reflexion

Das Praktikum war fachlich anspruchsvoll und abwechslungsreich zum lernen. Die Aufgabe ging über einfache Oberflächenanpassungen hinaus. Es förderte ein Verständnis für Datenflüsse, browserbasierte Bildverarbeitung, Schnittstellen zwischen Fenstern, geometrische Analyse und praktische Bedienbarkeit. Gleichzeitig war der medizinische Anwendungskontext motivierend, weil die technische Arbeit einen konkreten Dokumentationsnutzen hatte.

Herausfordernd war vor allem, dass Anforderungen schrittweise entstanden und teilweise erst im Testen, Besprechen und Probieren sichtbar wurde, welche Lösung wirklich sinnvoll ist. Ein Beispiel dafür ist die Nummerierung. Zunächst wurden Background-Segment-IDs betrachtet, später wurde klar, dass die sichtbare Nummerierung auf den gezeichneten Annotationen benötigt wird, um Summary, Bild und Daten eindeutig zu verbinden.

Positiv war die enge Betreuung und die Möglichkeit, Zwischenergebnisse regelmäßig zu besprechen. Dadurch konnten fachliche Missverständnisse früh erkannt und behoben werden. Gleichzeitig entstanden durch die Gespräche neue Perspektiven, etwa die spätere Verwendung der Daten für Verlaufsauswertungen, Avatar-Darstellungen oder 3D-Visualisierungen.

Aus meiner Sicht ist das Ergebnis des Praktikums ein stabiler funktionaler Stand, der als Grundlage für weitere Arbeiten genutzt werden kann. Das FATTA Tool kann Annotationen erfassen, nummerieren, mit Background-Segmenten verknüpfen, lesbar zusammenfassen und an ein aufrufendes Formular zurückgeben. Dies könnte ein wichtiger Baustein für eine weiterführende Bachelorarbeit oder für eine Integration in eine klinische Demonstrationsumgebung sein.

Für das weitere Studium und den Berufseinstieg hat das Praktikum verdeutlicht, wie wichtig die Verbindung aus technischer Umsetzung, fachlicher Kommunikation und sauberer Dokumentation ist. Gerade im medizinischen Umfeld reicht ein technisch funktionierender Prototyp allein nicht

aus. Entscheidend ist, dass die Daten nachvollziehbar, überprüfbar, wiederverwendbar und für Anwender verständlich sind.